

FORSETI

Three payment rails each approved ₹4,000. Every guard said yes. The ₹10,000 budget is now ₹2,000 overdrawn — and nobody broke a rule on their own road.

Dashboard :3005

API :8000

Seed 42

15 pages

6 executable attack vectors

12 AI agents

TRADITIONAL CONTROLS ASK

"Is this transaction valid on this rail?"

FORSETI ASKS

"Is this agent still acting within the authority it was given — across every rail?"

01

The idea, in plain language

You hire a robot assistant to buy groceries and tell it: *you may spend up to ₹10,000, on groceries*. That is a delegation — the robot now holds some of your spending authority.

Your money can leave your account through several different **rails** — different roads a payment can travel. A card rail. A UPI instant-transfer rail. An agentic machine-to-machine rail. Each road has its own guard, and each guard only watches their own road.

So a dishonest robot splits the job across all three.

CARD ₹4,000

APPROVED

UPI ₹4,000

APPROVED

AGENTIC ₹4,000

APPROVED

WHAT FORSETI ADDS UP

+₹2,000

CEILING ₹10,000

Each guard compared ₹4,000 to *its own* ₹10,000 limit and correctly said yes. The rule that broke is the one **nobody was watching**: your total. FORSETI is the missing watcher — it stands above all three roads and compares the sum to what you actually authorised.

02

Start it

Two processes, two terminals.

```
# Terminal 1 – the brain (API + engine)
cd backend
python -m uvicorn app.main:app --port 8000 --host 0.0.0.0

# Terminal 2 – the dashboard
cd frontend
npm run dev -- -p 3005
```

Open <http://localhost:3005>. Bottom-left of the sidebar you should see a green pulsing dot reading **Live stream** — that means the dashboard is attached to the engine's event feed. If it says "Reconnecting", the backend isn't up yet.

To regenerate every measurement first: `python tasks.py all` (about 25 seconds).

03

The 60-second demo

Go to **Live Arena**, leave the limit at 10000, keep **Cross-Rail Split** selected, press **EXECUTE ATTACK** — then don't touch anything for fourteen seconds.

- 
- A vertical timeline on the left side of the page, consisting of a light blue line with white circular markers at each event point.
- 0.5s **PLANNING CROSS RAIL SPLIT: ₹12,000 across 3 rails**
Red agent picks its vector and targets
- 1.3s **STEP 1: SENDING ₹4,000 VIA CARD**
Transaction 1 built and dispatched
- 2.2s **LOCALLY APPROVED — LOOKS NORMAL**
Card adapter checked *its own* limit only. Tile turns blue, shows ₹4,000
- 2.7s **AGGREGATING ALL RAILS: ₹4,000 / ₹10,000**
DTL adds it to the global running total
- 3.7s **ML RISK 6.8%**
Trained XGBoost scores it — low, and correctly so
- 6.4s **GLOBAL EXPOSURE ₹8,000 / ₹10,000**
Second rail approved. Meter turns amber
- 8.9s **AGGREGATING ALL RAILS: ₹12,000 / ₹10,000**
Third rail approved. The total now exceeds the grant
- 9.4s **VIOLATION: INV_01_GLOBAL_BUDGET_EXCEEDED**
The invariant fires — over by ₹2,000
- 11.3s **CONTAINING WITHOUT LOCKING THE USER OUT**
Cost governor picks partial authorisation, not a blanket block
- 12.6s **SIGNING DTL SNAPSHOT (ML-DSA-44)**
Post-quantum signature over the audit state
- 13.5s **NEXT STRATEGY: INTENT LAUNDERING**
Red agent observed the block and re-scored its options

NOTICE THIS, AND DON'T SKIP PAST IT

The ML risk score is only **6.8%**. That is not a bug and we do not hide it. A single ₹4,000 grocery payment genuinely *is* innocent-looking — no classifier can catch this from one transaction. Arithmetic on the total catches it. That gap is the entire argument of the project.

The measured result

Cross-rail splitting is **withheld from training entirely**, then tested. Recall on that unseen family — every model scored on the same test slice, seed 42.

ARCHITECTURE	CROSS-RAIL RECALL	READING
Rules only	0.257	Mostly blind
Per-rail ML (siloed)	0.000	Cannot see other rails
Global ML, no DTL features	0.000	Scores in isolation
Hybrid ML + DTL features	0.000	Still per-transaction
Deterministic DTL invariant	0.905	Compares the aggregate

This is the whole argument — and it is a negative result about our own classifier. Every learned model scores a transaction in isolation, so one ₹4,000 leg looks exactly like ordinary grocery spending. It is not detectable per-transaction, no matter how good the model is.

The DTL invariant compares *aggregate* exposure to the delegated ceiling, so it is holdout-independent: it never needed to see the attack family before, because it is arithmetic rather than pattern recognition. We report this rather than tuning it away. It is the reason the DTL exists.

MEASUREMENT	VALUE	SOURCE ARTIFACT
Model	XGBoost 3.4.1	evaluation/metrics.json
PR-AUC (temporal test)	0.8882	evaluation/metrics.json
ROC-AUC	0.9825	evaluation/metrics.json
Calibration ECE before → after	0.0204 → 0.0063	evaluation/metrics.json
Measured DTL feature lift	+0.1378 PR-AUC	evaluation/ablation_results.json
Inline pipeline p99 latency	0.879 ms	benchmark/latency.json
ML-DSA-44 tamper cases passed	4 / 4	live on every page load

WHAT WE ARE NOT CLAIMING

Fidelity Lab reads **NOT RUN / DATASET UNAVAILABLE**, and that is correct. PaySim and the ULB credit-card dataset are licensed and are not redistributed with this repository. The harness is real and executes; the anchor data simply isn't there, so *no realism claim is made*. Nothing anywhere is filled in with a plausible substitute.

05

The AI agent layer

Twelve agents that handle the work a payments team inherits the moment agentic spending ships. All of it is advisory.

THE RULE THAT MAKES THIS SAFE

The language model **never enforces**. It explains, translates and proposes; every proposal is schema-validated and re-checked by the deterministic engine before it can affect anything. Pull the API keys out and the security system behaves exactly as before — only the explanations disappear. Ten providers, sixty keys, tier-ordered fallback; running out of quota is a normal event, not an error.

Intent Compiler

01

Problem: a delegation is only as strong as its machine-readable form. “Nothing resellable” — the part you cared about — is thrown away at that boundary.

- Compiles plain words into ceiling, MCCs, exclusions, TTL
- Hallucinated categories are dropped, not silently widened
- Lists every ambiguity it had to resolve

Semantic Cart Auditor

02

Problem: merchant category codes are far too coarse. A real supermarket selling ₹7,800 of gift cards is compliant on paper.

- Judges economic substance, not the category code
- Splits legitimate from suspicious value
- Live: drift 0.9, ₹220 legit / ₹7,800 suspicious

Event Explainer

03

Problem: dashboards show that a control fired, never why. Click any log line in the arena.

- What / how / why the actor / why it matters
- Labelled Red or Blue

Adversarial Strategist

04

Problem: defences are graded against the attacks their authors imagined, which over-rates them.

- Proposes parameters, never outcomes
- The simulator executes and judges

- Falls back to a deterministic template from the event's own numbers

- The simulator executes and judges

Incident Report Writer

05

Problem: every contained incident creates a reporting obligation, written by hand from raw logs.

- Timeline, root cause, controls that fired *and did not*
- Writes "not established" rather than speculating

Policy Advisor

06

Problem: after an incident the reflex is "tighten everything", quietly destroying legitimate conversion.

- Smallest sufficient change
- States its false-positive cost explicitly
- Nothing applies without an operator

Customer Notice Writer

07

Problem: a held agent purchase reaches the customer as "Transaction declined", which destroys trust faster than the fraud does.

- SMS, in-app and email copy
- Never blames the customer

Regulatory Mapper

08

Problem: a novel control has no commercial value until a bank can map it to a duty it already carries.

- RBI / NPCI / PCI / FIU obligations
- Omits clause numbers it is unsure of, and lists gaps

Merchant Risk Profiler

09

Problem: MCC is self-declared and rarely re-checked, so attackers shop for a clean-looking category.

- Live: "FreshMart Grocery & Voucher Hub" declared grocery → inferred **quasi-cash**, action RESTRICT

Counterfactual Analyst

10

Problem: "would a lower limit have stopped this?" is normally answered by intuition in a meeting.

- The model proposes ceilings; the **simulator re-runs the attack** against each
- Live: proposed ₹3,999 → confirmed CONTAINED

Log Copilot

11

Problem: nobody can write a query language during a live incident.

- Compiles English into a filter the engine runs
- Never answers from memory

Model Card Generator

12

Problem: model documentation is a regulatory expectation and always written last, by someone motivated to flatter the model.

- Built from real artifacts
- Rejects any card with no stated weaknesses

Every page, every control

Fifteen pages. Every sidebar entry leads somewhere real.

Overview

[/](#)

The pitch. Thesis, live authority, system health, and the headline cross-rail table.

- Any card reading NOT RUN means that pipeline hasn't executed

Live Arena ★

[/arena](#)

The judging page. The attack streamed live, with the DTL interception.

- Delegated limit** — type any ceiling, press APPLY
- DTL defense** toggle — OFF is the legacy world
- Speed** 0.5× / 1× / 2×
- Vector checkboxes** — select several to run a campaign
- EXECUTE ATTACK / RESET**
- Click any log line to explain that step

Attack Simulator

[/simulator](#)

All 52 researched vectors with citations. Filter by channel, surface, severity, agentic relevance.

- 6 marked Executable launch for real; 46 are Research only

Defense Center

[/defense](#)

The three invariants with their formal expressions, and the graduated response ladder.

- ALLOW → STEP_UP → PARTIAL_AUTH → QUARANTINE → ... → BLOCK

Transaction Monitor

[/transactions](#)

One row per transaction with the two columns that matter side by side: rail verdict vs DTL verdict.

- APPROVED beside CONTAINED is the contradiction

Delegation Ledger

[/ledger](#)

Two-phase exposure: settled, authorized, pending, reserved — plus scope and manual limit control.

- Counting in-flight spend is what closes the race window

Agents

[/agents](#)

The closed loop. Red's strategy scoring table and Blue's policy adaptation.

- Scores derive from observed outcomes, not an if/else chain

Threat Intelligence

[/threat-intel](#)

Coverage by channel and attack surface, plus the agentic-only vectors.

Detection Lab[/detection](#)

The science. PR-AUC, baselines under both conditions, ablation, family holdout.

- Every panel shows its experiment ID and seed

Fidelity Lab[/fidelity](#)

KS, correlation distance, discriminator AUC, TSTR — or an honest NOT RUN.

- Currently NOT RUN: licensed anchors absent

Explainability[/explainability](#)

Real `shap.TreeExplainer`, global and per-transaction.

- Red pushes toward fraud, green toward legitimate

Policy Center[/policy](#)

Seven delegation policies, which is active, and every Blue adaptation triggered by a real violation.

Quantum Audit[/audit](#)

The signed snapshot, its hash, and four tamper cases executed live.

- Two buttons let you verify and then break a signature yourself

Replay & Demo[/replay](#)

Play, pause, step and re-run any recorded round from its JSONL log at original timing.

System Settings[/settings](#)

Environment, package versions, artifact status with regeneration commands, and the data-safety constraints.

TRY THIS – THE MOST PERSUASIVE 30 SECONDS

In Live Arena, turn **DTL defense OFF** and attack. Every rail approves, exposure reaches ₹12,000 against a ₹10,000 grant, nothing objects, and the round ends *RED TEAM WINS*. That is today's world. Toggle it back ON and run the identical attack — contained. Then set the limit to **₹20,000** and run again: ₹12,000 now fits, so no violation fires. The rule is arithmetic on your number, not a threshold we tuned.

07

What happens to one transaction

Eight stages, in order. Each emits structured events that are simultaneously logged, written to JSONL for replay, and broadcast over the WebSocket.

- 1 `simulator/adapters/` **Rail adapter** — checks only its own limit and merchant scope. Approves.
- 2 `dtl/ledger.py` **DTL ledger** — adds it to global exposure across every rail.
- 3 `dtl/invariant_engine.py` **Invariant engine** — is `settled + authorized + pending + reserved + new_tx ≤ ceiling` ? If not, emits a machine-checkable proof.
- 4 `detector/feature_schema.py` **Feature extractor** — 29 features. The same code runs in training and here, so there is zero train/serve skew.
- 5 `detector/inference.py` **ML detector** — loads the trained artifact from disk. Never trains at startup.
- 6 `dtl/cost_governor.py` **Cost governor** — separates legitimate from suspicious value and picks the mildest sufficient action.
- 7 `crypto/mldsa_audit.py` **PQC auditor** — canonicalises the DTL state and signs it with ML-DSA-44.
- 8 `feedback/adaptive_planner.py` **Feedback engine** — Red rescores its strategies; Blue hardens policy.

WHY THE PICTURE CAN'T LIE

The dashboard holds no independent state. Every arrow, label and amount is rendered from an event the backend actually emitted — which is why the event log beside the diagram works as a receipt. Stop the backend and the arena goes idle rather than animating.